AI Productivity and AI API Integration for Developers

Organized by Hacktiv8

This Program is part of the AI Opportunity Fund: Asia Pacific,
in collaboration with AVPN and supported by Google.org and the Asian Development Bank.

HACKTIV8

Pembuatan Chatbot berbasis Gemini AI Model

Sesi 3



HACKTIV8

Sesi 1

Pengenalan AI & Membuat *Website* Menggunakan AI

- Konsep AI, generative AI & etikanya
- Tekning prompting untuk *Large Language Model* (LLM)
- Membangun *Website* menggunakan Gemini Canvas & V0
- Perangkat AI untuk produktivitas
  - Gemini Code Assist
  - Copilot
  - Google Antigravity
- Pengenalan NotebookLM

Sesi 2

Eksplorasi Gemini AI API

- Pengenalan Gemini AI API
- Menginisialisasi model Gemini AI
- Menghasilkan teks dengan input:
  - Gambar
  - Audio
  - Teks
  - dokumen
- File API

Sesi 3

Pembuatan Chatbot berbasis Gemini AI Model

- Mengkonfigurasi Gemini AI API
- Parameter (temp, top_p, top_k)
- *System Instruction*
- Membuat *Chatbot*
- Mengimplementasikan Gemini AI pada *Chatbot*

HACKTIV8

Sesi 1

Pengenalan AI & Membuat *Website* Menggunakan AI

- Konsep AI, generative AI & etikanya
- Tekning prompting untuk *Large Language Model* (LLM)
- Membangun *Website* menggunakan Gemini Canvas & V0
- Perangkat AI untuk produktivitas
  - Gemini Code Assist
  - Copilot
  - Google Antigravity
- Pengenalan NotebookLM

Sesi 2

Eksplorasi Gemini AI API

- Pengenalan Gemini AI API
- Menginisialisasi model Gemini AI
- Menghasilkan teks dengan input:
  - Gambar
  - Audio
  - Teks
  - dokumen
- File API

Sesi 3

Pembuatan Chatbot berbasis Gemini AI Model

- Mengkonfigurasi Gemini AI API
- Parameter (temp, top_p, top_k)
- *System Instruction*
- Membuat *Chatbot*
- Mengimplementasikan Gemini AI pada *Chatbot*

HACKTIV8

Pembuatan Chatbot berbasis Gemini AI Model

Agenda

- Memahami dasar-dasar membangun chatbot
- Memahami cara mengintegrasikan model AI Gemini untuk mendukung respons chatbot
- Memahami Jenis parameter dalam konfigurasi Gemini (temperature, top_p, top_k)
- Memahami cara konfigurasi parameter di Gemini
- Membangun chatbot sederhana menggunakan HTML, CSS, dan JavaScript



Absensi

HACKTIV8

Isi Absensi Dulu, Ya!



https://bit.ly/absensi-developers

HACKTIV8

Persiapan Awal

Checklist

- Sudah terinstall Node.js v18+
- Akun Google Cloud dengan Gemini API
- Gemini API key
- Sudah terinstall VS Code
- Starter Code Front End dengan Vanilla JS (Periksa lampiran pada kode.id di bawah ini)

Instalasi Node.js v18+

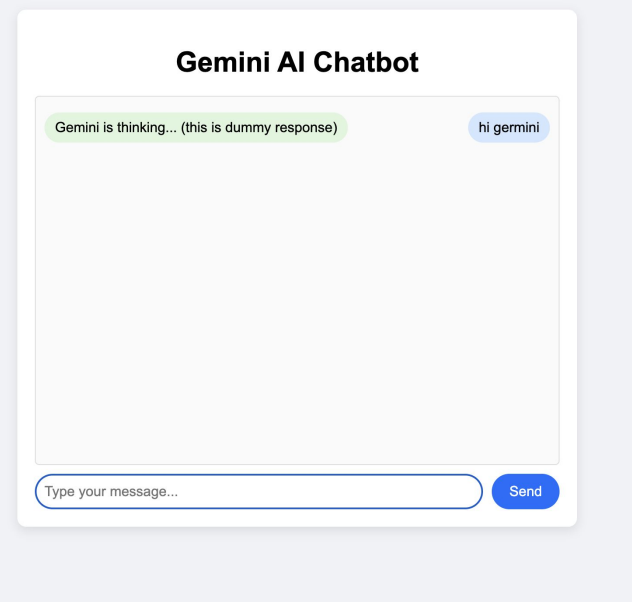
Installation guide

```
hacktiv8@Air-Hacktiv8 Sesi3 % node -v
v23.7.0
```

Persiapan Awal

Starter Code Vanilla JS

Download zip yang berisikan file html, css, and js.
Debug starter-code ke browser seperti ini:
https://bit.ly/startercode-developers



HACKTIV8

Pratinjau Proyek

Definisi Chatbot

Chatbot adalah aplikasi yang dirancang untuk mensimulasikan percakapan dengan pengguna manusia.

Chatbot dapat bekerja berdasarkan:

- Aturan yang telah ditentukan (decision tree, pernyataan if sederhana)
- Machine learning atau large language model (seperti Gemini)

Chatbot banyak digunakan dalam:

- Layanan pelanggan (customer service)
- Asisten virtual
- Sistem rekomendasi
- Alat produktivitas pribadi (misalnya Copilot, ChatGPT, Google Bard)



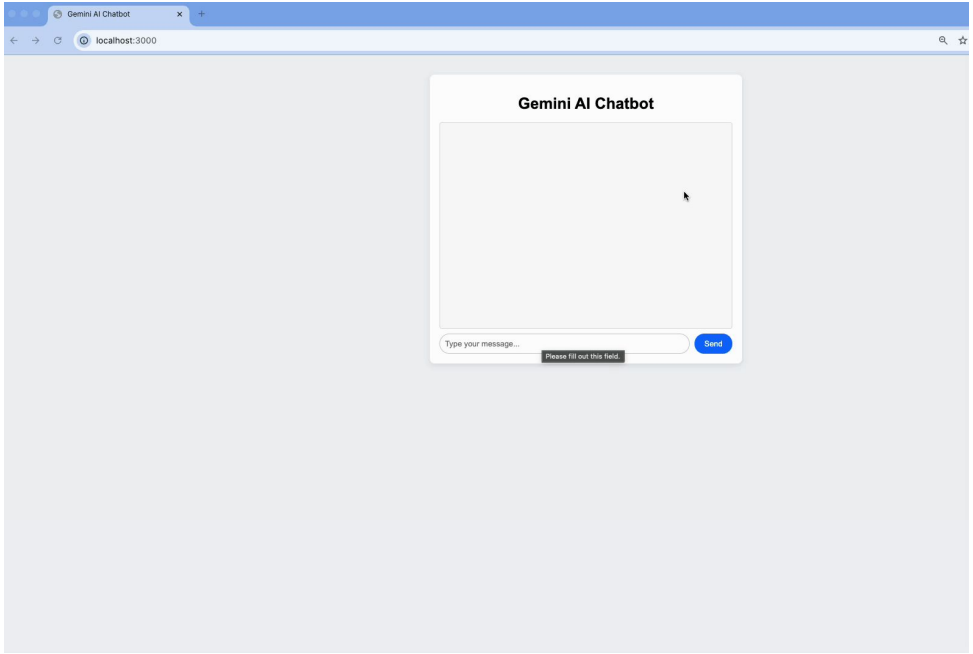
Persiapan Proyek

Dalam sesi ini, kita akan membangun **chatbot berbasis web sederhana** yang dapat:

- Menerima input teks dari pengguna (melalui browser)
- Mengirim pesan tersebut ke server (Node.js)
- Meminta **Gemini AI** untuk menghasilkan respons yang relevan
- Menampilkan pesan hasil generate AI kembali ke pengguna secara **real-time**

Ini **bukan chatbot statis (hardcoded)** — chatbot ini akan menghasilkan balasan cerdas secara dinamis, didukung oleh model **Google Gemini 2.5 Flash**.

Hasil yang Diharapkan



Chatbot ini mensimulasikan percakapan langsung — bukan dengan logika acak — tetapi dengan meminta Gemini untuk berpikir dan merespons secara alami. Ini adalah cara praktis yang bagus untuk mempelajari cara kerja API AI dalam aplikasi dunia nyata.

Cara Mengintegrasikan Model AI Gemini untuk Pembuatan Chatbot.

HACKTIV8

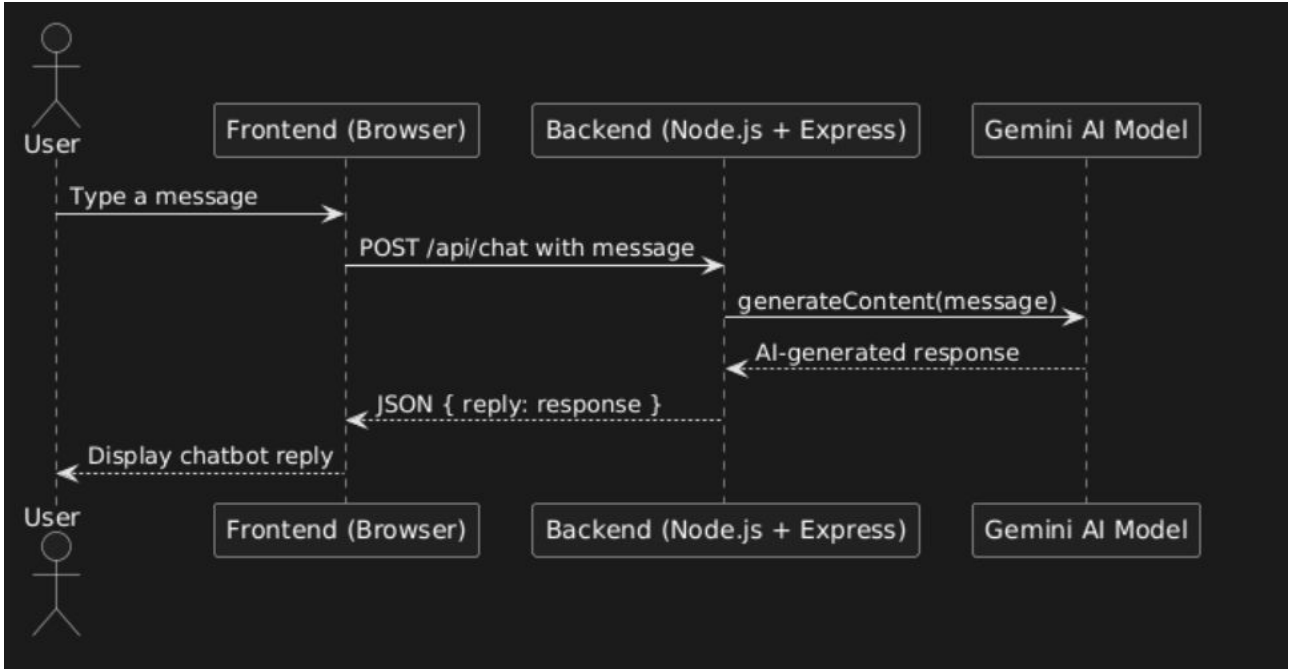
Pendahuluan

Dalam pengembangan web modern, integrasi AI ke dalam aplikasi menjadi semakin mudah berkat tools seperti **Google Gemini API**. Model **Gemini AI** mampu menghasilkan respons yang kaya konteks, natural, dan menyerupai percakapan manusia berdasarkan prompt dari pengguna. Ketika diterapkan pada sistem chatbot, model ini menggantikan logika berbasis aturan tradisional dengan respons cerdas secara real-time yang dapat beradaptasi dengan setiap input pengguna.

Proses integrasi umumnya melibatkan tiga langkah utama:

1. **Mengambil input** dari pengguna di sisi frontend.
2. **Mengirim input** tersebut ke layanan backend.
3. Menggunakan model AI untuk **menghasilkan respons** dan mengembalikannya ke pengguna.

Alur Kerja Chatbot



HACKTIV8

Alur Kerja Chatbot

Dalam proyek praktik ini, kita menerapkan alur tersebut menggunakan **frontend Vanilla JavaScript** dan **backend Node.js + Express**. Saat pengguna mengirim pesan melalui form chat, frontend akan mengirim **request POST** ke endpoint **/api/chat** di backend. Backend kemudian memanggil metode **generateContent()** dari **Gemini SDK** untuk meneruskan pesan pengguna ke model AI. Setelah Gemini menghasilkan respons, hasil tersebut dikirim kembali ke frontend dan ditampilkan di antarmuka chat.

Arsitektur ini memungkinkan logika AI tetap berada di sisi **backend**—sehingga lebih **aman dan scalable**—sekaligus memberikan pengalaman percakapan yang mulus bagi pengguna.

Absensi

HACKTIV8

Buat yang belum absen, absen yuk! WAJIB



https://bit.ly/absensi-developers

Konfigurasi Gemini

HACKTIV8

Jenis-Jenis Parameter pada Konfigurasi Gemini

Gemini mendukung **penyesuaian parameter** yang membantu **membentuk cara model merespons.**

Parameter	Purpose	Value range
temperature	Mengontrol keacakan dalam output. Nilai lebih tinggi = lebih kreatif.	0.0 – 2.0
top_k	Membatasi respons pada top-K token yang paling mungkin	1 – 40
top_p	Menggunakan nucleus sampling untuk membatasi tingkat keacakan	0.0 – 1.0

Mengapa ini penting?

Bayangkan kamu sedang membangun sebuah API yang membutuhkan berbagai “kepribadian”:

- Untuk **penulisan kreatif**, kamu mungkin ingin menggunakan **temperature: 0.9** agar hasilnya lebih variatif dan imajinatif.
- Untuk **tanya jawab faktual**, nilai yang lebih rendah seperti **0.2** membantu memastikan jawaban lebih akurat dan presisi.

System Instruction pada Konfigurasi Gemini

System Instruction (sering juga disebut *System Prompt* atau *System Message*) adalah instruksi level tinggi yang diberikan kepada model AI untuk menetapkan konteks, perilaku, dan batasan sebelum model tersebut mulai berinteraksi dengan pengguna.

System Instruction tidak terlihat oleh pengguna akhir (user), tetapi sangat mempengaruhi bagaimana AI menjawab. Fungsinya meliputi:

- **Menetapkan Persona:** Memberi tahu AI "siapa" dirinya.
  - *Contoh:* "Kamu adalah asisten pemrograman Python senior yang teliti."
- **Mengatur Nada Bicara (Tone):** Menentukan gaya bahasa.
  - *Contoh:* "Jawablah dengan gaya yang santai, humoris, dan menggunakan emoji." atau "Jawablah dengan formal dan akademis."
- **Memberikan Batasan (Constraints):** Memberi tahu apa yang **tidak** boleh dilakukan.
  - *Contoh:* "Jangan pernah memberikan nasihat medis," atau "Jawab hanya dalam Bahasa Indonesia."
- **Mengatur Format Output:** Menentukan bentuk jawaban yang diinginkan.
  - *Contoh:* "Selalu berikan jawaban dalam format JSON," atau "Sertakan tabel dalam setiap perbandingan."

Ada yang mau sharing

rencana kustomisasi chatbot?



Step#1
Membangun API

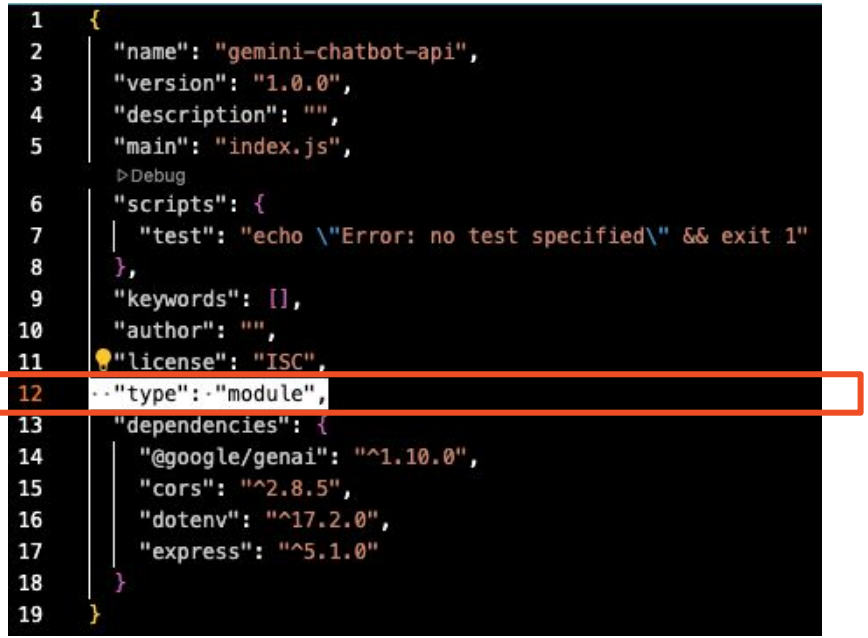
Set Up Proyek

```
$ mkdir gemini-chatbot-api
$ cd gemini-chatbot-api
$ npm init -y
$ npm install express dotenv cors @google/genai
```

Setup ini menyiapkan project **Node.js** menggunakan **Express** dan mengintegrasikan **Gemini 2.5 Flash API** melalui package @google/genai, sehingga mendukung penanganan input berupa teks, audio, gambar, maupun dokumen.

- express: Menyiapkan REST API.
- dotenv: Memuat API key Gemini secara aman dari file .env.
- @google/genai: Menghubungkan aplikasi ke Gemini API (termasuk Gemini 2.5 Flash).
- cors: Mengizinkan request dari berbagai origin (domain) untuk mengakses endpoint pada server.

Set Up Proyek



```
{
  "name": "gemini-chatbot-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "@google/genai": "^1.10.0",
    "cors": "^2.8.5",
    "dotenv": "^17.2.0",
    "express": "^5.1.0"
  }
}
```

Untuk menggunakan ES Modules dengan syntax import/export di dalam projek Node.js, tambahkan "type": "module" di dalam package.json.

Struktur File



Highlight:

.env : file ini menyimpan ***environment variables***, terutama kredensial yang sensitif seperti Gemini API Key. contoh:

```
GEMINI_API_KEY=your_credential_key
```

package.json : Mendefinisikan *dependency* dan *scripts* untuk aplikasi Node.js.

index.js :
Berisi seluruh logika untuk REST endpoint yang berinteraksi dengan **Gemini AI**, meliputi:

- Menyiapkan aplikasi Express
- Menghubungkan ke model **Gemini 2.5 Flash**
  Mendefinisikan *endpoint* untuk menerima input teks, gambar, dokumen, dan audio
- Menangani *parsing request, upload file,* manajemen error, dan *response*

index.js

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import { GoogleGenAI } from '@google/genai';

const app = express();

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

const GEMINI_MODEL = "gemini-2.5-flash";

app.use(cors());
app.use(express.json());

const PORT = 3000;
app.listen(PORT, () => console.log(`Server ready on http://localhost:${PORT}`));
```

Kode ini menyiapkan server Express dasar yang terintegrasi dengan Gemini AI API (model 2.5 Flash):

- **Memuat environment variables** menggunakan dotenv, termasuk Gemini API key.
- **Mengimpor modul penting**: Express untuk membangun REST API, CORS untuk mengizinkan *request* lintas *origin*, dan Gemini AI SDK untuk mengakses generative AI dari Google.
- **Menginisialisasi aplikasi Express** dan *client* Gemini AI menggunakan API key.
- **Mendefinisikan variabel global** untuk model Gemini default agar mudah diganti di satu tempat.
- **Menerapkan middleware** untuk menangani CORS dan *parsing request* JSON.
- **Menjalankan server** pada port yang ditentukan (default: 3000), siap melayani *endpoint* berbasis Gemini.

Endpoint POST api/chat

```
app.post('/api/chat', async (req, res) => {
    const { conversation } = req.body;
    try {
        if (!Array.isArray(conversation)) throw new Error('Messages must be an array!');

        const contents = conversation.map(({ role, text }) => ({
            role,
            parts: [{ text }]
        }));

        const response = await ai.models.generateContent({
            model: GEMINI_MODEL,
            contents,
            config: {
                temperature: 0.9,
                systemInstruction: "Jawab hanya menggunakan bahasa Indonesia.",
            },
        });
        res.status(200).json({ result: response.text });
    } catch (e) {
        res.status(500).json({ error: e.message });
    }
})
```

Endpoint ini memungkinkan percakapan *multi-turn* dengan Gemini AI:

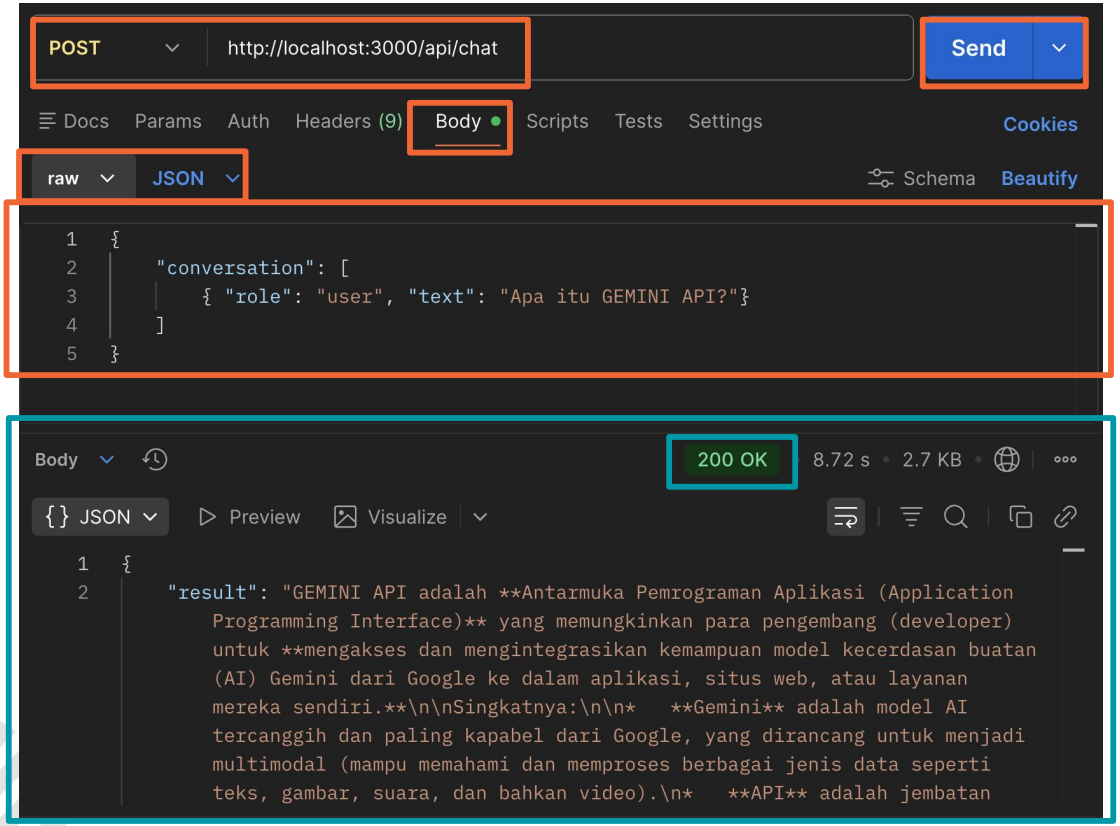
- **Menerima request** POST ke endpoint /api/chat dengan *array messages* di *body request* yang merepresentasikan riwayat percakapan (role dan pesan).
- **Memvalidasi** bahwa *input* berupa *array*; jika tidak, *server* akan mengembalikan *error*.
- **Mengubah setiap pesan** ke format yang kompatibel dengan Gemini (role dan parts).
- Mengkonfigurasi jawaban Gemini dengan temperature dan systemInstruction
- **Mengirim pesan** yang telah diformat ke model Gemini menggunakan generateContent() dengan model yang ditentukan pada GEMINI_MODEL.
- **Mengembalikan respons AI** melalui properti response.text.
- **Menangani error** dengan mengembalikan status 500 dan pesan error jika proses gagal.

Pengujian API

Jalankan Aplikasi:

```
hacktiv8@MacBook-Air-Hacktiv8 gemini-chatbot-api % node index.js
Gemini Chatbot running on http://localhost:3000
```

Pengujian API



Langkah-Langkah:

- Buka Postman
- **Method**: POST
- **URL**: http://localhost:3000/api/chat
- **Tab**: **Body → raw → JSON**
- Masukkan prompt
- Klik Send

Absensi

HACKTIV8

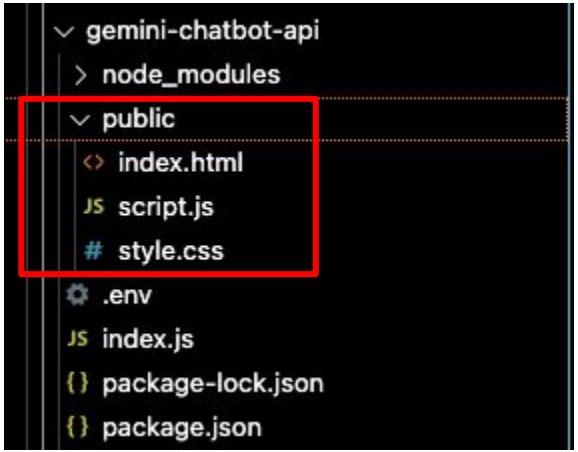
Buat yang belum absen, absen yuk! WAJIB



https://bit.ly/absensi-developers

Step#2
Menghubungkan API dengan Front End

Struktur File



Highlight:

Dalam struktur ini, folder **public** berfungsi sebagai *entry point* frontend untuk project **gemini-chatbot-api**. Folder ini berisi:

- **index.html** – File HTML utama yang dirender di browser.
- **script.js** – Menangani logika frontend, seperti mengirim input pengguna ke Gemini API melalui HTTP request.
- **style.css** – Mengatur tampilan UI untuk antarmuka chatbot.

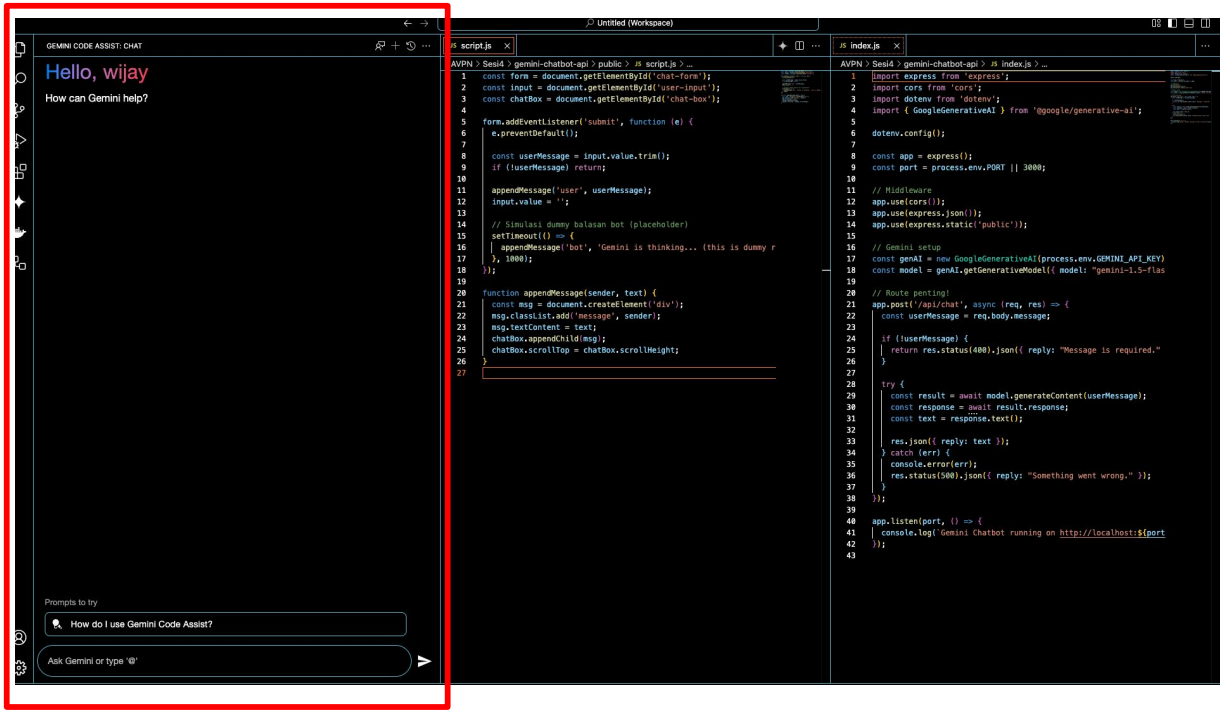
Dengan menempatkan kode Vanilla JavaScript di dalam folder **public**:

- Aset statis (HTML, JS, CSS) terpisah dari logika backend.
- ExpressJS dapat menyajikan aset-aset tersebut menggunakan middleware seperti **express.static**.

Struktur ini memungkinkan kamu untuk:

- Menjalankan backend Gemini API menggunakan **Node.js / Express**.
- Mengakses frontend chatbot melalui **http://localhost:3000/** setelah server dijalankan.

Gemini Code Assist



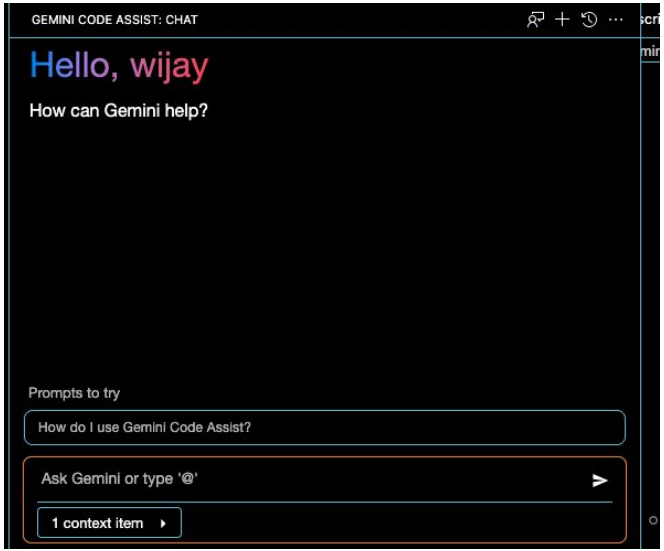
Dalam sesi ini, kita akan mengintegrasikan Gemini Code Assist ke dalam frontend menggunakan **Vanilla JavaScript (plain JS)**.

Tujuannya adalah menghubungkan file **script.js** di folder **public/** dengan endpoint API backend **/api/chat** untuk menciptakan pengalaman chatbot yang interaktif.

Gemini Code Assist

Pada langkah ini, kita akan menggunakan Gemini Code Assist untuk membantu menulis kode JavaScript yang diperlukan guna menghubungkan form frontend dengan backend API.

- Buka panel chat Gemini Code Assist di project kamu.
- Pada kolom input, salin dan tempel prompt berikut, lalu tekan Enter:



Gemini Code Assist - Prompt

```
I'm building a chatbot web app using Node.js + Express (backend) and Vanilla JavaScript (frontend).
My backend exposes a POST endpoint at /api/chat that expects a JSON body in this format:

{
  "conversation": [
    { "role": "user", "text": "<user_message>" }
  ]
}

The backend uses Google Gemini API to generate AI responses and returns a JSON object like:

{ "result": "<gemini_ai_response>" }

On the frontend, I have a form with a text input and a submit button. When the user submits the form, I want to:
- Add the user's message to the chat box.
- Show a temporary "Thinking..." bot message.
- Send the user's message as a POST request to /api/chat (with JSON body above).
- When the response arrives, replace the "Thinking..." message with the AI's reply (from the `result` property).
- If an error occurs or no result is received, show "Sorry, no response received." or "Failed to get response from server."

Can you help me write the complete script.js for the frontend (Vanilla JS, no frameworks) that covers this flow, including
proper error handling and DOM manipulation? Please make sure the code is simple and production-ready.

The HTML structure is:
<form id="chat-form">
  <input type="text" id="user-input" />
  <button type="submit">Send</button>
</form>
<div id="chat-box"></div>

Please provide the complete code for script.js only, matching the backend API spec above.
```

Gemini Code Assist - Prompt



Prompt



Responses

Gemini Code Assist - Responses



Coba solusi kode dari Gemini Assist ke dalam **script.js**

Gemini Code Assist

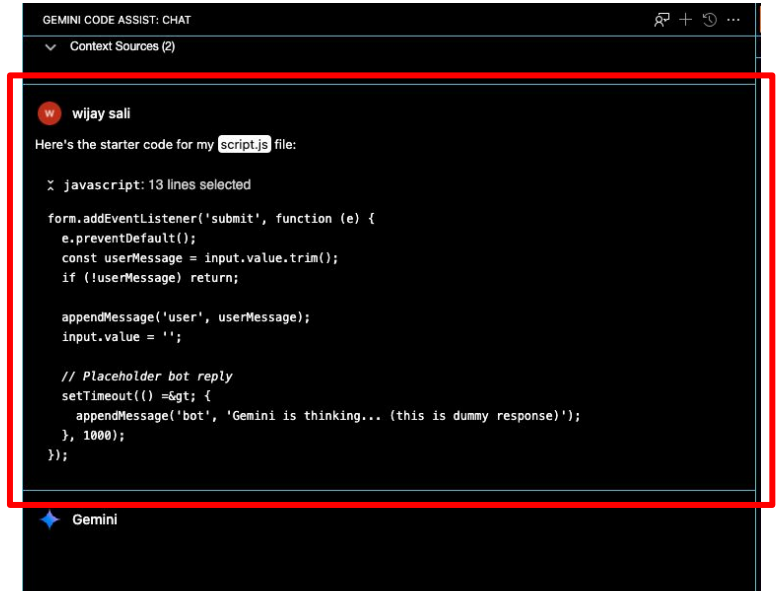
Jelaskan dimana meletakkan **fetch** di dalam kode frontend

````
Here's the starter code for my `script.js` file:

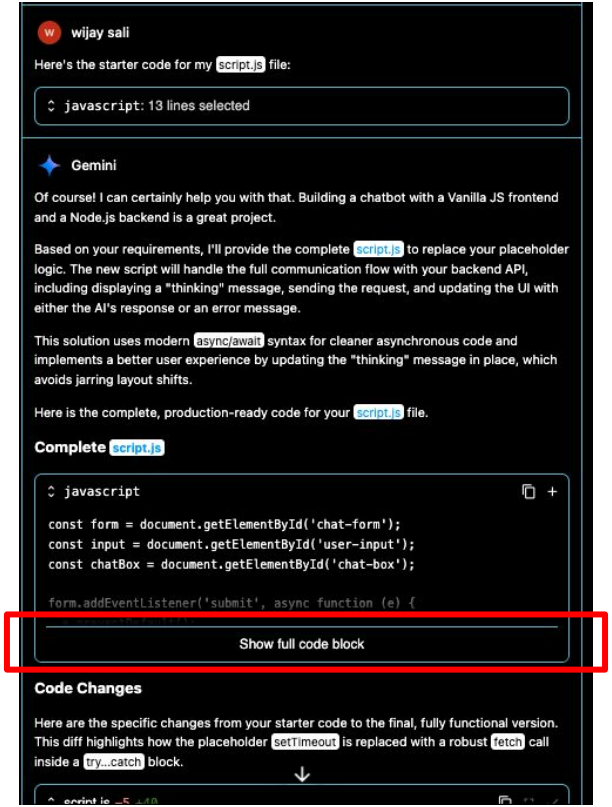
```javascript
form.addEventListener('submit', function (e) {
  e.preventDefault();
  const userMessage = input.value.trim();
  if (!userMessage) return;

  appendMessage('user', userMessage);
  input.value = '';

  // Placeholder bot reply
  setTimeout(() => {
    appendMessage('bot', 'Gemini is thinking... (this is
dummy response)');
  }, 1000);
});
````


````

Gemini Code Assist - Responses



Gemini Code Assist - Responses

Apa yang kita pelajari dari Gemini:

Gemini menjelaskan bahwa setelah mengirim request ke backend /api/chat, sebaiknya frontend terlebih dahulu menampilkan pesan sementara seperti *"Gemini is thinking..."*. Pesan ini nantinya akan diperbarui dengan respons AI yang sebenarnya.

Setelah respons diterima, Gemini menyarankan untuk mengecek apakah data yang dikembalikan memiliki *property* result.

- Jika ada, nilai dari result (jawaban AI) ditampilkan sebagai balasan chatbot.
- Jika tidak ada, tampilkan pesan cadangan seperti *"Sorry, no response received."*.

Pendekatan ini membuat penanganan respons API lebih *robust*, serta memastikan antarmuka pengguna selalu memberikan *feedback* yang jelas, baik saat AI berhasil memberikan jawaban maupun saat terjadi *error* atau respons kosong.

```
form.addEventListener('submit', async function (e) {
  e.preventDefault();

  const userMessage = input.value.trim();
  if (!userMessage) return;

  appendMessage('user', userMessage);
  input.value = '';

  // Create a placeholder message that we can update later
  const botMessageElement = document.createElement('div');
  botMessageElement.classList.add('message', 'bot');
  botMessageElement.textContent = 'Gemini is thinking...';
  chatBox.appendChild(botMessageElement);
  chatBox.scrollTop = chatBox.scrollHeight;

  try {
    const response = await fetch('/api/chat', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        messages: [{ role: 'user', content: userMessage }],
      }),
    });

    if (!response.ok) {
      // Try to get a more specific error from the server's JSON response
      const errorData = await response.json().catch(() => null); // Gracefully handle non-JSON error responses
      const errorMessage = errorData?.error || response.statusText;
      throw new Error(`Server error: ${errorMessage}`);
    }

    const data = await response.json();

    if (data && data.result) {
      botMessageElement.textContent = data.result;
    } else {
      botMessageElement.textContent = 'Sorry, no response received.';
    }
  } catch (error) {
    console.error('Error fetching response:', error);
    botMessageElement.textContent = 'Failed to get response from server.';
  } finally {
    // Ensure we scroll to the bottom after the final message is rendered
    chatBox.scrollTop = chatBox.scrollHeight;
  }
```

Menghubungkan kode index.js

Sebelum melakukan testing atau menjalankan kode chatbot frontend yang diberikan oleh Gemini, pastikan index.js (backend) kamu sudah mengaktifkan CORS dan menyiapkan file statis frontend (HTML, JS, CSS)

Hal ini penting agar frontend dapat berkomunikasi dengan backend API dan dimuat di browser tanpa masalah CORS.

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import path from 'path';
import { fileURLToPath } from 'url';
import { GoogleGenAI } from "@google/genai";

// ==== Tambahan setup __dirname untuk ESM (import style) ====
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();

const ai = new GoogleGenAI({ apiKey: process.env.API_KEY });
const GEMINI_MODEL = "gemini-2.5-flash";

app.use(cors());
app.use(express.json());

// ==== Tambahan middleware untuk serve file static (frontend) ====
// Serve all files in public_solution (HTML, JS, CSS) at root path
app.use(express.static(path.join(__dirname, 'public')));
```

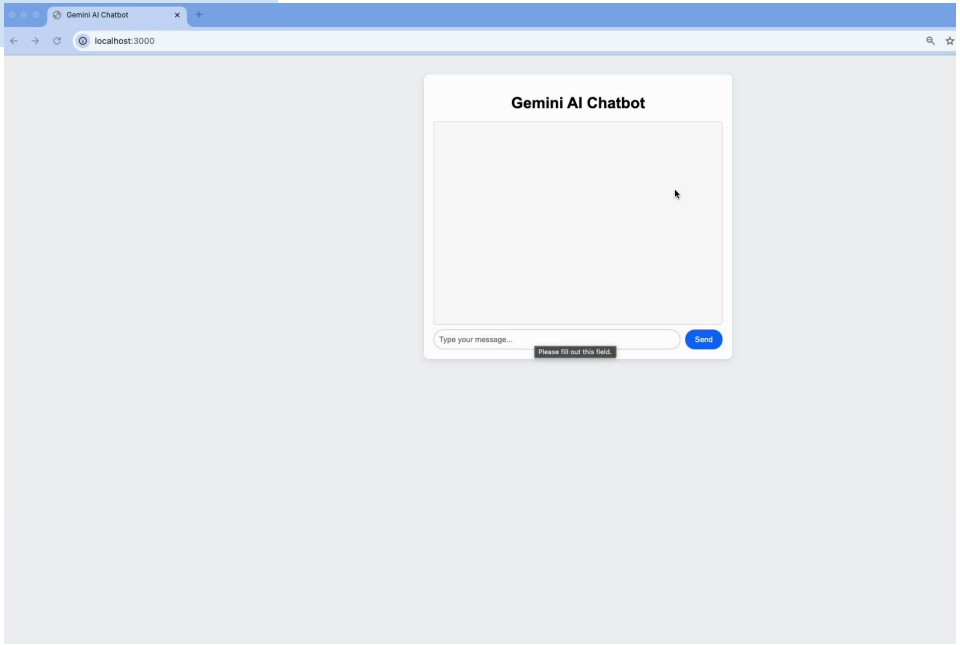
Pengujian API

Jalankan Aplikasi:

```
hacktiv8@MacBook-Air-Hacktiv8 gemini-chatbot-api % node index.js
Gemini Chatbot running on http://localhost:3000
```

Pengujian Front End

Jalankan Aplikasi:



Chatbot ini mensimulasikan percakapan secara *real-time* bukan dengan logika acak, melainkan dengan meminta **Gemini** untuk bernalar dan merespons secara natural. Ini merupakan cara praktik yang sangat baik untuk memahami bagaimana **AI API** bekerja dalam aplikasi dunia nyata.

Tips untuk Menggunakan Gemini Secara Efektif

- Jaga prompt tetap singkat, tetapi tetap berikan konteks yang cukup.
- Jelaskan dengan jelas di mana kode akan ditempatkan (misalnya di dalam event listener).
- Gunakan prompt lanjutan seperti ***"Dapatkah anda mengoptimisasi kode ini?"*** atau ***"Tolong wrap kode ini ke dalam satu function."*** untuk penyempurnaan kode.



Instruksi Pengumpulan Proyek

Instruksi

Pembuatan Proyek

Buatlah sebuah chatbot berbasis AI dengan use case dan konfigurasi parameter yang sesuai dengan kreativitas masing-masing. Chatbot harus menggunakan model AI (misalnya NLP/LLM) untuk memproses bahasa alami dan memberikan respon yang relevan kepada pengguna.

- **Contoh Use Case**: customer service bot, education bot, travel assistant, personal productivity assistant, dll.
- **Contoh Parameter Kreatif**: gaya bahasa (formal/santai), domain pengetahuan tertentu (misalnya kesehatan, edukasi, hobi), integrasi API eksternal, atau fitur tambahan seperti memory dan rekomendasi.

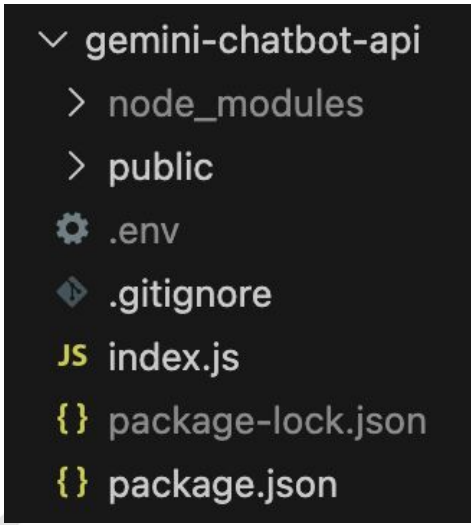
Deliverables:

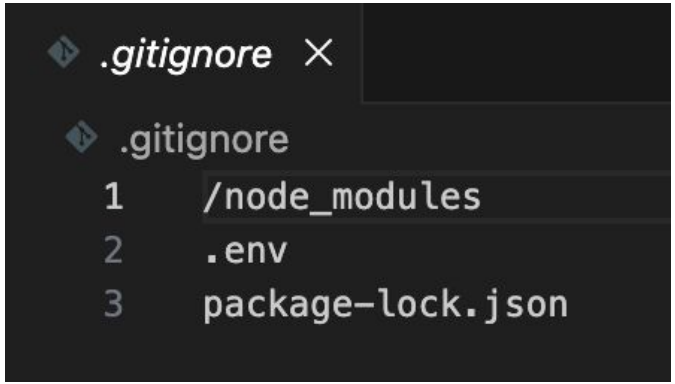
- URL repositori GitHub
- Screenshoots User Interface

Instruksi

Pengumpulan Proyek

Untuk mengumpulkan projek Anda dengan benar, Anda perlu mempublikasikan folder lokal Anda ke *repository* GitHub. Sebelum mempublikasikannya, Anda perlu membuat file baru **.gitignore** untuk mengecualikan file dan folder yang tidak akan kita publikasikan ke GitHub





/node_modules dan **package-lock.json** dikecualikan karena folder dan file ini dapat di generate ulang dari file package.json dengan npm install.
.env dikecualikan karena file ini berisi gemini api key yang sifatnya rahasia

Instruksi

Pengumpulan Proyek

Untuk mengumpulkan projek Anda dengan benar, Anda perlu mempublikasikan folder lokal Anda ke *repository* GitHub. Ikuti langkah-langkah berikut:

```
git init // Inisialisasi Git di dalam folder projek
git add . // Menambahkan semua file ke area staging
git commit -m "Implementasi endpoint Gemini AI API" // Commit pertama dengan isi yang jelas
git branch -M main // Mengubah nama branch saat ini menjadi main
git remote add origin https://github.com/yourusername/your-repo-name.git //Hubungkan ke remote repo
git push -u origin main // Push kode ke repository GitHub
```

Setelah push ke GitHub:

1. **Salin (copy) URL repository GitHub**
   (misalnya: *https://github.com/yourusername/gemini-ai-api-project*)
2. **Tempelkan (paste) URL** ke **form pengumpulan projek** yang ada **di akhir sesi**

Instruksi

Pengumpulan Proyek

HACKTIV8

Due Date:
H+2 Sesi 5 | 23.59 WIB



https://bit.ly/finalproject-developers

Kesimpulan

Kesimpulan

Dalam sesi ini, kita mempelajari cara membangun chatbot berbasis AI yang sederhana namun *powerful* menggunakan **Vanilla JavaScript** di sisi frontend dan **Node.js dengan Express** di sisi backend. Dengan mengintegrasikan **Google Gemini AI**, chatbot mampu menghasilkan respons cerdas secara **real-time**, melampaui chatbot tradisional yang berbasis aturan statis.
Proyek hands-on ini memperkenalkan peserta pada dasar-dasar **arsitektur chatbot**, komunikasi **full-stack melalui API**, serta cara meningkatkan aplikasi dengan **model AI modern**. Melalui proses pengembangan bertahap—mulai dari desain UI hingga integrasi backend—peserta merasakan langsung alur kerja nyata dalam membangun aplikasi web interaktif.

Poin Penting (Key Takeaways):

- Chatbot mensimulasikan percakapan antara pengguna dan mesin.
- Gemini AI memungkinkan respons yang natural dan dinamis melalui API.
  fetch() menghubungkan frontend dan backend secara asynchronous.
  Route Express menangani pesan pengguna dan meneruskannya ke Gemini.
- Proyek ini menjembatani UI, logika backend, dan integrasi AI dalam satu alur kerja terpadu.

Quiz 3

Quiz 3

HACKTIV8

Due Date:
Hari ini, 23.59 WIB

Ingat, absensi juga dihitung dari pengisian Quiz.



https://bit.ly/quiz3-developers

Post-Test



https://bit.ly/posttest-developers

Bagaimana cara mengisi Post-Test?

a. Isilah form Post-Survey ini terlebih dahulu: https://bit.ly/mba2-postsurvey
b. Screenshots bukti apabila sudah menyelesaikan Post-Survey
c. Submit bukti tersebut ke dalam link Post-Test: https://bit.ly/posttest-developers
d. Lanjutkan mengisi Post-Test sampai selesai

- **Country**: Indonesia
- **Local Training Partner (LTP)**: PT. Hacktivate Teknologi Indonesia (Hacktiv8)

HACKTIV8

Terima Kasih

Hacktiv8
www.hacktiv8.com

(021) 8067 5787
halo@hacktiv8.com

Hacktiv8id

Hacktiv8id

Hacktiv8

Hacktiv8 Indonesia